



Supervisory Control of a Storm Surge Barrier System

4SC080 , Supervisory control of cyber
physical systems - Q3 (2026)

Group 03

Full Name	Student ID
Thomas Leenderts	1552333
Rick Schneider	1459279
Jens van Esch	1843370



Eindhoven, April 16, 2026

Contents

1	Introduction	1
2	Description of abstract finite state model	2
2.1	Phase automaton	2
2.2	Sea automaton	2
2.3	Gate actuators	3
2.4	Fjord water level	4
2.5	Water flow	5
2.6	Rainwater	6
2.7	Ships	6
3	Requirements	8
3.1	G1: Gates must close when the sea water level is above 4.5 [m], to prevent flooding of the land behind	8
3.2	G2: Gates are opened at most at an intermediary height of 3 [m] when the water level in the fjord is not near critically high or low levels	8
3.3	G3: To prevent peak load on the electricity net, gate actuation (starting to open or close) has an interval time of 5 [s]	9
3.4	L1: The water level in the fjord should not exceed 4.0 [m]	9
3.5	L2: The water level in the fjord should not fall below 1.0 [m]	9
3.6	S1: Ships will only enter the canal if the flow generated by the storm surge barrier is small enough (below 100 [m ³ /s])	9
3.7	S2: Ships should be allowed into the canal once in a while (stopping the ships from entering for eternity is not an acceptable solution)	10
3.8	H1: Hydraulic operating policy near critical fjord levels	10
3.9	H2: Conservative safeguard near the upper fjord bound	11
4	Test sequences	13
4.1	Test sequence 1: Gate closure enforced by rising sea level	13
4.2	Test sequence 2: Gate opening enforced by high fjord during falling sea	13

1 | Introduction

I have read the TU/e Code of Scientific Conduct. In teaching activities (including group assignments), I will respect the five central values of scientific integrity, namely: trustworthiness, intellectual honesty, openness, independence and social responsibility, as well as the standards and principles that follow from them.

Signed:

Thomas Leenderts - 1552333 - 16/04/2026

Rick Schneider - 1459279 - 16/04/2026

Jens van Esch - 1843370 - 16/04/2026

With rising sea levels, coastal regions face an increasing risk of flooding. To protect low-lying areas, countries have constructed storm surge barriers and sluice systems that regulate water flow between the sea and inland waterways. Because failures of such critical water infrastructure can have severe socioeconomic consequences, reliable control of these systems is essential.

This report addresses the supervisory control of a storm surge barrier system inspired by the barrier at *Hvide Sande*, Denmark, as part of the STORM SAFE project. The barrier separates the *Ringkøbing Fjord* from the North Sea through a canal with multiple vertically actuated gates.

The goal of this project is to design a supervisory controller for this system using the synthesis-based engineering approach. This involves abstracting the physical system into a discrete finite-state model under simplifying assumptions, formulating formal requirements, synthesizing a supervisor that is guaranteed to satisfy all requirements, and validating the resulting controlled system through simulation. This report further explains the implementation of this synthesis-based controller.

The report is structured as follows. Section 2 describes the abstract finite-state plant model and its components. Section 3 presents the informal and formal requirements. Section 4 provides test sequences that demonstrate the presence of required behavior and the absence of forbidden behavior in the controlled system.

2 | Description of abstract finite state model

2.1 | Phase automaton

To structure the interaction between supervisory decisions and the environment, a separate phase automaton was introduced. The purpose of this automaton is to alternate between a *decision phase*, in which the supervisor may choose control actions, and an *environment phase*, in which one abstract environmental time step is executed.

The automaton has two locations: **Decide** and **Env**. In location **Decide**, all controllable gate commands are enabled, i.e., opening and closing commands for each individual gate, as well as the ship-signal commands `c_ship_green` and `c_ship_red` are allowed to occur. In addition, the uncontrollable rain events `u_start_rain` and `u_stop_rain` remain possible. The gate-step events `Gatei.u_gate_step` are also enabled in this phase, because once a gate has been instructed to start moving, the actual execution of that movement step is part of the plant behavior and is therefore modeled as uncontrollable.

The transition from **Decide** to **Env** is given by the controllable event `c_end_cycle`. This transition is only enabled when no gate is still waiting for its uncontrollable start step, expressed by the guard

$$\text{no_gate_start_pending.}$$

This prevents the supervisor from ending the decision phase while a gate is still in the intermediate **Opening** or **Closing** location.

In location **Env**, the model executes exactly one environmental step. Rain events are still allowed here, after which the uncontrollable event `u_sea_step` transfers the automaton back to **Decide**. As a result, the model follows a cyclic structure:

$$\text{Decide} \rightarrow \text{Env} \rightarrow \text{Decide} \rightarrow \dots$$

This decomposition makes the model easier to interpret and ensures a clear separation between supervisory choices and the abstract 3-minute evolution of the environment. Without this automaton at any point in time the uncontrollable time step can occur leading to only bad states and unintended behavior after synthesis. This abstraction can best be interpreted as the controller being allowed to evaluate the system state once every 3 minutes after which a change in state can be made, assuming this state evaluation and decision making happens in a comparably insignificant amount of time compared to the 3 minute interval.

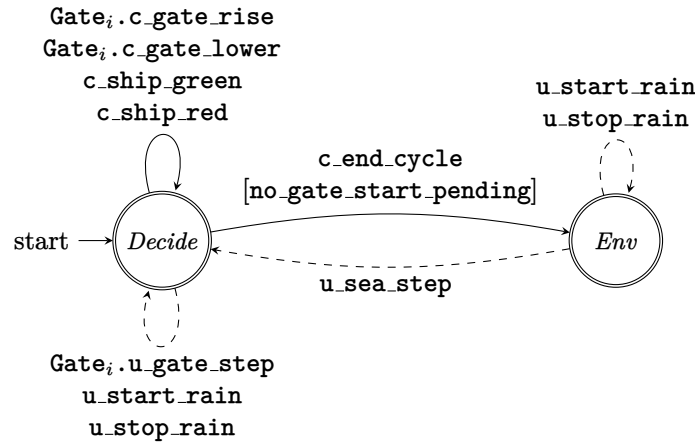


Figure 2.1: Phase automaton. Dashed edges denote uncontrollable events. The *Decide* location contains supervisory decisions, while *Env* executes one environmental step before returning to *Decide*.

2.2 | Sea automaton

The assignment models the sea water level as a sinusoidal tide with amplitude $A = 2.5$ m and two tidal cycles per day. In the adopted abstraction, the environment evolves in steps of 3 minutes. Therefore, instead of evaluating the sine function continuously, the sea tide is sampled at a fixed interval of

$$\Delta t = 180 \text{ s.}$$

Since the tide repeats every 12 hours, one period contains

$$\frac{12 \cdot 60}{3} = 240$$

discrete samples. This leads to a compact finite-state representation using a counter

$$k \in \{0, 1, \dots, 239\}.$$

In the CIF model, the sea automaton consists of a single location and a discrete variable k . At every occurrence of the uncontrollable event `u_sea_step`, the counter is updated according to

$$k := (k + 1) \bmod 240.$$

The actual sea water level is not stored as a separate automaton location, but is computed algebraically from k through a precomputed lookup table (in a switch case for synthesis-purposes):

$$l_s = T[k].$$

Here, $T[k]$ contains the sampled tide values in millimetres, ranging from 0 mm to 5000 mm. The initial value is chosen as

$$k = 153, \quad l_s = 4401 \text{ mm},$$

which corresponds to a challenging condition which will show the functionality of our supervisor after a little amount of events invoked after initialization.

This modeling choice has two advantages. First, it preserves the intended periodic tidal behavior from the assignment while remaining fully discrete. Second, it avoids the need for continuous variables or run-time trigonometric evaluation during synthesis. The resulting model is therefore deterministic, finite-state, and directly compatible with the data-based supervisory synthesis workflow.

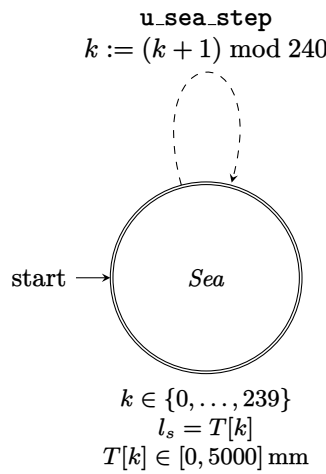


Figure 2.2: Sea automaton. Dashed edges denote uncontrollable events. The counter k stores the tide phase, while the sea level l_s is obtained from the lookup table $T[k]$.

2.3 | Gate actuators

Each gate of the storm surge barrier is vertically actuated by a 3-phase induction motor connected to a winch system. The physical opening height ranges from 0 to 6 m, with an intermediary position at 3 m. Full actuation from fully closed to fully open takes 6 minutes. The change in height is governed by the actuator input $u_i(t)$ [m/s]:

$$\dot{h}_i = u_i(t)$$

where $u_i(t)$ is positive when gate i is raising, negative when lowering, and 0 otherwise. The barrier consists of 14 identical gates; for this assignment, 4 are modeled.

To reduce the state space for CIF synthesis, the continuous height is abstracted to three discrete positions:

$$h \in \{0, 1, 2\} \longleftrightarrow \{0 \text{ m}, 3 \text{ m}, 6 \text{ m}\}$$

Each unit step of h corresponds to 3 m of physical gate travel. This abstraction captures the three operationally relevant positions (closed, intermediary, fully open) while keeping the number of states small enough for data-based supervisory control synthesis.

Each gate is modeled using a **plant def** automaton definition, which allows all four gate instances **Gate1**–**Gate4** to be created from a single definition. This ensures modularity and makes it straightforward to scale the model to more gates in the future. The automaton has four locations and a discrete variable $h \in [0, 2]$ that tracks the abstracted height.

The gate starts in the initial and marked location *Idle*. In this location, the supervisor may issue a controllable **c_gate_rise** command (when $h < 2$) or a controllable **c_gate_lower** command (when $h > 0$). If no command is given, the gate synchronises on **u_sea_step** and remains in *Idle*.

When a rise or lower command is issued, the gate moves to the *Opening* or *Closing* location respectively. In these locations, the gate waits for the uncontrollable event **u_gate_step**, which models the 5 s start-up delay required by specification G3. Upon this event, h is incremented or decremented by 1, and the gate transitions to location *Moving*.

In the *Moving* location, the gate waits for the next **u_sea_step** event, after which it returns to *Idle*. This completes one 3-minute time step. All four locations are marked to ensure the supervisor does not block the gate from completing its motion cycle, which would violate the nonblocking property.

Full actuation from $h = 0$ to $h = 2$ requires two consecutive rise commands across two time steps, totalling $2 \times 3 = 6$ minutes, consistent with the physical specification.

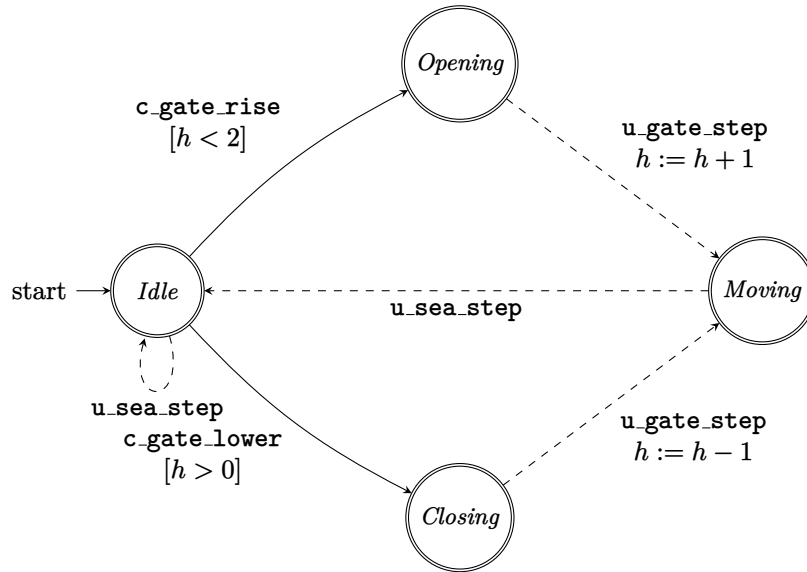


Figure 2.3: Gate actuator automaton. Dashed edges denote uncontrollable events. The variable h represents the abstracted gate height ($0 = 0 \text{ m}$, $1 = 3 \text{ m}$, $2 = 6 \text{ m}$).

2.4 | Fjord water level

The water level of the fjord l_f [m] is influenced by rainfall and by water exchange through the gates. In continuous form, this can be written as

$$\dot{l}_f = \frac{1}{A_f} \left(R(t) \pm \sum_{i=1}^n Q_i(t) \right),$$

where A_f [m²] is the effective fjord area, $R(t)$ [m³/s] is the rain inflow, and $Q_i(t)$ [m³/s] is the flow through gate i . The sign of the gate contribution depends on the relative sea and fjord levels: if the sea level is higher, water enters the fjord through the gates; if the fjord level is higher, water leaves the fjord through the gates.

In the CIF model, the fjord water level is represented by the discrete variable `lvl` with range [900, 4100] in millimetres. The initial value is `lvl = 2500` mm. This interval covers the operating region used by the plant and supervisor with some room for potential requirement violation to ensure requirement violations is possible.

The level is updated every 3 minutes on the uncontrollable event `u_sea_step`. The plant uses three mutually exclusive algebraic conditions, `pos_flow`, `neg_flow`, and `no_flow`, to determine the flow direction:

1. When the sea level is higher than the fjord level ($l_s > l_f$), water flows from the sea into the fjord:

$$\text{lvl}' = \text{lvl} + \text{rain_flow} + \text{gate_flow}$$

2. When the fjord level is higher than the sea level ($l_s < l_f$), water flows from the fjord to the sea:

$$\text{lvl}' = \text{lvl} + \text{rain_flow} - \text{gate_flow}$$

3. When both levels are equal ($l_s = l_f$), there is no net gate flow:

$$\text{lvl}' = \text{lvl} + \text{rain_flow}$$

This structure gives a direct discrete approximation of the intended hydraulic behaviour while keeping the update expressions simple enough for data-based synthesis.

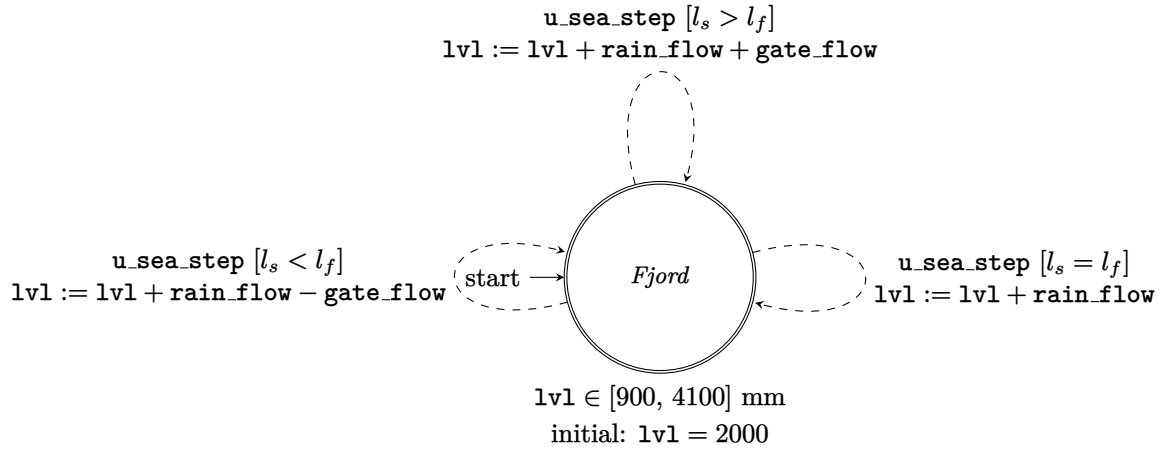


Figure 2.4: Fjord water level automaton. The single location has three uncontrollable self-loops on `u_sea_step`, selected by the relative sea and fjord levels.

2.5 | Water flow

The gate flow is modeled using the abstract gate heights:

$$Q_i = c \cdot h_i,$$

where h_i is the opening height of gate i and the effective sign of the flow depends on the relative sea and fjord levels. A larger opening produces a larger water exchange through the gates.

In the CIF model, the total gate opening is defined as

$$\text{total_open} = \sum_{i=1}^4 h_i, \quad h_i \in \{0, 1, 2\}.$$

Since there are four gates, `total_open` ranges from 0 to 8. This total opening is converted into a fjord level change per 3-minute step through the coefficient `gate_coeff`:

$$\text{gate_flow} = \text{total_open} \times \text{gate_coeff}, \quad \text{gate_coeff} = 20.$$

In the implemented abstraction, `gate_flow` is expressed in [mm]/[3 min] and represents the change in fjord water level caused by the combined gate openings during one environmental step. It is therefore not interpreted as the physical gate discharge Q_i in m^3/s , but as its discrete effect on the fjord level. The sign of this effect is determined separately in the fjord automaton from the relative sea and fjord levels.

The resulting `gate_flow` therefore ranges from 0 to 160 mm per step. The variable itself is always non-negative; the fjord automaton determines whether this contribution is added or subtracted from `lvl`.

The same aggregate quantity is also used in the supervisory requirements to bound the total hydraulic effect of the gates during ship passage. In this way, the model links the abstract gate positions directly to the safety constraints on canal operation.

2.6 | Rainwater

Rainfall is modeled as a separate plant automaton with two locations: *Dry* and *Rain*. The automaton starts in *Dry*, and both locations are marked. This gives a compact binary abstraction of weather conditions while remaining sufficient for the hydraulic behaviour included in the model.

The rain contribution to the fjord is defined algebraically by

$$\text{rain_flow} = \begin{cases} 10, & \text{when the automaton is in location } \textit{Rain}, \\ 0, & \text{when the automaton is in location } \textit{Dry}. \end{cases}$$

Rain therefore adds a fixed inflow of 10 mm to the fjord on each `u_sea_step` while the system is in the raining state.

The transitions between *Dry* and *Rain* are given by the uncontrollable events `u_start_rain` and `u_stop_rain`. These events are uncontrollable because rainfall is an environmental process and cannot be influenced by the supervisor.

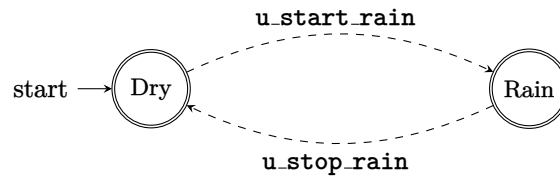


Figure 2.5: Rain automaton. Rainfall is represented as an uncontrollable switch between the locations *Dry* and *Rain*.

2.7 | Ships

To control the ships through the fjord, 1 ship signal is modeled. This signal indicates whether ships are allowed to pass or it is not safe to cross.

A ship model consists of two locations: *red* and *green*. The automaton starts in the initial location *red*, indicating that ships are not allowed to pass. The location *green* represents the situation where ships are permitted to pass and is marked.

Transitions between these locations are controlled by controllable events. The event `c_ship_green` switches the signal from *red* to *green*, allowing ships to pass. The event `c_ship_red` switches the signal from *green* back to *red*. These events are controllable because the supervisor determines when it is safe to allow ships

to pass, based on the abstract barrier-induced fjord level change and the corresponding hydraulic conditions.

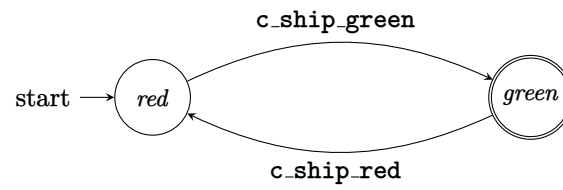


Figure 2.6: Ship signal automaton

3 | Requirements

Requirements are used to describe what the system is allowed to do and what it is not allowed to do. They are important for synthesis, because they guide the supervisor in controlling the system.

In this project, the requirements are mainly used to prevent unsafe or unwanted situations. For example, it is required to make sure that the green light is not green when the flow is unsafe, or that certain actions are only allowed when the system is in a safe state.

These requirements are combined with the plant model during synthesis. The synthesis tool then generates a supervisor that blocks controllable events whenever a requirement would be violated, while still allowing as much normal behavior as possible.

3.1 | G1: Gates must close when the sea water level is above 4.5 [m], to prevent flooding of the land behind

This requirement ensures that all gates are closed when the sea water level becomes too high. In the model, this is implemented by an invariant that cannot have a gate with a height above zero when the sea level exceeds 4.5 m.

Because the model uses an abstraction, a pre-close zone is also used. In this zone, gates are no longer allowed to start opening, so that they can still close in time before the critical sea level is reached. The values are found from the discretization of the sinusoidal sea profile

```

1 alg bool sea_close_zone = sea_level.current_water_level > 4500;
2 alg bool sea_preclose_zone = sea_level.current_water_level >= 4443;
3
4 requirement invariant not (sea_close_zone and Gate1.h > 0);
5 requirement invariant not (sea_close_zone and Gate2.h > 0);
6 requirement invariant not (sea_close_zone and Gate3.h > 0);
7 requirement invariant not (sea_close_zone and Gate4.h > 0);
8
9 requirement req_no_rise_in_preclose:
10     location Ok:
11         initial;
12         marked;
13
14         edge Gate1.c_gate_rise when not sea_preclose_zone;
15         edge Gate2.c_gate_rise when not sea_preclose_zone;
16         edge Gate3.c_gate_rise when not sea_preclose_zone;
17         edge Gate4.c_gate_rise when not sea_preclose_zone;
18 end

```

3.2 | G2: Gates are opened at most at an intermediary height of 3 [m] when the water level in the fjord is not near critically high or low levels

This requirement limits the gate opening when the fjord water level is in a noncritical range. In the model, the intermediary height of 3 m corresponds to $h = 1$. Therefore, when the fjord level is not near a critical low or high value, the gates may not open further than this intermediary position.

The value of near critical high is set at 3.5 meters and near critical low is set at 1.5 meters.

```

1 alg bool fjord_near_low = fjord_water_level.lvl <= 1500;
2 alg bool fjord_near_high = fjord_water_level.lvl >= 3500;
3 alg bool fjord_noncritical = not (fjord_near_low or fjord_near_high);
4
5 requirement Gate1.c_gate_rise needs not (fjord_noncritical and Gate1.h >= 1);
6 requirement Gate2.c_gate_rise needs not (fjord_noncritical and Gate2.h >= 1);
7 requirement Gate3.c_gate_rise needs not (fjord_noncritical and Gate3.h >= 1);
8 requirement Gate4.c_gate_rise needs not (fjord_noncritical and Gate4.h >= 1);

```

3.3 | G3: To prevent peak load on the electricity net, gate actuation (starting to open or close) has an interval time of 5 [s]

This requirement ensures that there is always a delay of 5 seconds between two gate-actuation starts. In the model, this is represented by the requirement automaton *gate_start_interval*. After one gate starts rising or lowering, the automaton moves to location *Wait*. Only when the uncontrollable event *u_gate_step* occurs, the model returns to *Ready*, after which a new actuation may start.

```

1 requirement gate_start_interval:
2     location Ready:
3         initial;
4         marked;
5
6         edge Gate1.c_gate_rise goto Wait;
7         edge Gate1.c_gate_lower goto Wait;
8         edge Gate2.c_gate_rise goto Wait;
9         edge Gate2.c_gate_lower goto Wait;
10        edge Gate3.c_gate_rise goto Wait;
11        edge Gate3.c_gate_lower goto Wait;
12        edge Gate4.c_gate_rise goto Wait;
13        edge Gate4.c_gate_lower goto Wait;
14
15        edge u_sea_step;
16        edge u_start_rain;
17        edge u_stop_rain;
18        edge c_ship_green;
19        edge c_ship_red;
20
21    location Wait:
22        marked;
23
24        edge Gate1.u_gate_step goto Ready;
25        edge Gate2.u_gate_step goto Ready;
26        edge Gate3.u_gate_step goto Ready;
27        edge Gate4.u_gate_step goto Ready;
28
29        edge u_start_rain;
30        edge u_stop_rain;
31        edge c_ship_green;
32        edge c_ship_red;
33 end

```

3.4 | L1: The water level in the fjord should not exceed 4.0 [m]

This requirement ensures that the water level in the fjord remains below the upper limit of 4.0 m. It is modeled directly as an invariant on the fjord water level.

```

1 requirement invariant fjord_water_level.lvl <= 4000;

```

3.5 | L2: The water level in the fjord should not fall below 1.0 [m]

This requirement ensures that the water level in the fjord does not drop below the lower limit of 1.0 m. Just like L1, it is modeled as an invariant.

```

1 requirement invariant fjord_water_level.lvl >= 1000;

```

3.6 | S1: Ships will only enter the canal if the flow generated by the storm surge barrier is small enough (below 100 [m³/s])

This requirement ensures that ships are only allowed to enter the canal when the barrier-induced hydraulic effect is sufficiently small. In the implemented abstraction, this is evaluated using the variable

`fjord_water_level.gate_flow`, which is expressed in [mm]/[3 min] and represents the corresponding fjord level change during one environmental step. A boolean condition `ship_flow_safe` is used to indicate whether this abstract flow effect remains below the chosen safety threshold. The event that changes the ship signal to green is only allowed when this condition holds. In addition, an invariant is imposed to ensure that the ship signal cannot be green while the flow condition is unsafe.

```

1 alg bool ship_flow_safe = abs(fjord_water_level.gate_flow) <= 40;
2
3 requirement c_ship_green needs ship_flow_safe;
4 requirement invariant not (ships.green and not ship_flow_safe);

```

3.7 | S2: Ships should be allowed into the canal once in a while (stopping the ships from entering for eternity is not an acceptable solution)

Due to the nature of the requirements as defined for S1 it means that as long as it is possible for the ships to safely enter the canal the controller can set the traffic light to green. No strict time requirement is set in the model as it would greatly increase synthesis cost without real improvement in the model abstraction.

3.8 | H1: Hydraulic operating policy near critical fjord levels

Besides the requirements stated in the assignment, an additional hydraulic policy was introduced to make the supervisor use favorable tide windows while remaining conservative in unfavorable situations. This policy is based on the fjord zones.

$$l_f \leq 1500 \text{ mm}, \quad l_f \geq 3500 \text{ mm},$$

The tide direction is derived from the sea counter k , in the implemented abstraction the sea is considered rising for:

$$60 \leq k < 180,$$

A hysteresis margin of 44 mm around the condition $l_s = l_f$. The hysteresis avoids switching between inflow and outflow decisions when the sea and fjord levels are nearly equal.

When the fjord is near its lower limit, opening the gates is only hydraulically useful if the sea is rising and sufficiently above the fjord level, such that inflow from sea to fjord can occur in the current abstraction. In all other low-water situations, further opening is restricted and the total opening is bounded by

$$\text{total_open} = \sum_{i=1}^4 h_i \leq 2.$$

Conversely, when the fjord is near its upper limit, opening is only useful if the sea is falling and sufficiently below the fjord level, such that drainage from fjord to sea is possible. In the remaining high-water situations, a stricter bound is used:

$$\text{total_open} \leq 1.$$

During favorable refill and drainage windows, the supervisor is not allowed to reduce the opening further. At the environmental step, at least a limited opening must remain available:

$$\text{total_open} \geq 2.$$

At the same time, each individual gate is limited to the intermediary position $h_i \leq 1$. In this way, the controller uses the available hydraulic opportunity, while avoiding unnecessarily aggressive gate openings. The values used for this (l_f and hysteresis margin) are derived from the steps in discretization of the sinusoidal sea profile.

```

1 alg bool sea_rising = sea_level.k >= 60 and sea_level.k < 180;
2 alg bool sea_falling = not sea_rising;
3
4 const int policy_margin = 44;
5
6 alg bool sea_well_above_fjord =
7     sea_level.current_water_level >= fjord_water_level.lvl + policy_margin;
8 alg bool sea_well_below_fjord =

```

```

9      sea_level.current_water_level <= fjord_water_level.lvl - policy_margin;
10
11 alg bool low_refill_now =
12     fjord_near_low and sea_rising and sea_well_above_fjord;
13 alg bool low_conserve_now =
14     fjord_near_low and not (sea_rising and sea_well_above_fjord);
15 alg bool high_drain_now =
16     fjord_near_high and sea_falling and sea_well_below_fjord;
17 alg bool high_protect_now =
18     fjord_near_high and not (sea_falling and sea_well_below_fjord);
19
20 requirement invariant not (
21     low_conserve_now and fjord_water_level.total_open > 2
22 );
23
24 requirement invariant not (
25     high_protect_now and fjord_water_level.total_open > 1
26 );
27
28 requirement invariant not (
29     low_refill_now and phase.Env and fjord_water_level.total_open < 2
30 );
31
32 requirement invariant not (
33     high_drain_now and phase.Env and fjord_water_level.total_open < 2
34 );
35
36 requirement invariant not (low_refill_now and Gate1.h > 1);
37 requirement invariant not (low_refill_now and Gate2.h > 1);
38 requirement invariant not (low_refill_now and Gate3.h > 1);
39 requirement invariant not (low_refill_now and Gate4.h > 1);
40
41 requirement invariant not (high_drain_now and Gate1.h > 1);
42 requirement invariant not (high_drain_now and Gate2.h > 1);
43 requirement invariant not (high_drain_now and Gate3.h > 1);
44 requirement invariant not (high_drain_now and Gate4.h > 1);
45
46 requirement Gate1.c_gate_rise needs not (low_conserve_now or high_protect_now);
47 requirement Gate2.c_gate_rise needs not (low_conserve_now or high_protect_now);
48 requirement Gate3.c_gate_rise needs not (low_conserve_now or high_protect_now);
49 requirement Gate4.c_gate_rise needs not (low_conserve_now or high_protect_now);
50
51 requirement Gate1.c_gate_lower needs not (low_refill_now or high_drain_now);
52 requirement Gate2.c_gate_lower needs not (low_refill_now or high_drain_now);
53 requirement Gate3.c_gate_lower needs not (low_refill_now or high_drain_now);
54 requirement Gate4.c_gate_lower needs not (low_refill_now or high_drain_now);

```

3.9 | H2: Conservative safeguard near the upper fjord bound

A second additional safeguard was introduced for the implemented synthesis model when the fjord level reaches

$$l_f \geq 3730 \text{ mm.}$$

From this threshold onward, all gates are forced back to the closed configuration. This restriction is not part of the original assignment specification, but was added as a conservative model-specific safeguard for the present three-minute discrete abstraction. Its purpose is to exclude problematic abstract states close to the hard upper bound $l_f \leq 4000$ mm, where the combination of environmental updates and uncontrollable rain events may otherwise lead to undesirable supervisory behavior. This values was chosen based on the maximum water height that can be added by the rain during high tide, otherwise a blocking state would be entered where no environmental steps could be taken as a result of the phase automaton.

```

1 alg bool fjord_force_high_close = fjord_water_level.lvl >= 3730;
2

```

```
3 requirement invariant not (fjord_force_high_close and Gate1.h > 0);  
4 requirement invariant not (fjord_force_high_close and Gate2.h > 0);  
5 requirement invariant not (fjord_force_high_close and Gate3.h > 0);  
6 requirement invariant not (fjord_force_high_close and Gate4.h > 0);
```

4 | Test sequences

To verify that the synthesized supervisor enforces the specified requirements, a collection of event sequences is presented below. Each sequence is executed in the interactive simulator of Eclipse ESCET, starting from the initial state ($k=153$, $lvl=2500$ mm, all gates closed, no rain, phase = *Decide*). At every step the simulator only offers transitions that the supervisor permits, confirming that the trace is a valid supervised execution.

For each sequence it is indicated whether it demonstrates the *presence* of relevant required behavior or the *absence* of relevant forbidden behavior.

4.1 | Test sequence 1: Gate closure enforced by rising sea level

This sequence tests the supervisor's response to an approaching storm. Starting just below the pre-close threshold, the supervisor is given the opportunity to open a gate. As the sea rises into the pre-close zone, the supervisor must close all open gates before the sea exceeds 4.5 m.

Table 4.1: Test sequence 1: Gate closure enforced by rising sea level.

#	Event	Effect	Sea	Fjord	h_1
0	—	Initial state	4401	2500	0
1	<code>Gate1.c_gate_rise</code>	Supervisor opens Gate 1; gate enters <i>Opening</i>	4401	2500	0
2	<code>Gate1.u_gate_step</code>	Gate 1 height becomes 1 (3 m); gate enters <i>Moving</i>	4401	2500	1
3	<code>c_end_cycle</code>	No pending gate starts; phase $\rightarrow$ <i>Env</i>	4401	2500	1
4	<code>u_sea_step</code>	$k \rightarrow 154$, sea = 4443 mm; Gate 1 $\rightarrow$ <i>Idle</i> ; fjord +20 $\rightarrow$ 2520 mm (positive gate flow)	4443	2520	1
5	<code>Gate1.c_gate_lower</code>	Supervisor lowers Gate 1; gate enters <i>Closing</i>	4443	2520	1
6	<code>Gate1.u_gate_step</code>	Gate 1 height becomes 0 (closed); gate enters <i>Moving</i>	4443	2520	0
7	<code>c_end_cycle</code>	Phase $\rightarrow$ <i>Env</i>	4443	2520	0
8	<code>u_sea_step</code>	$k \rightarrow 155$, sea = 4483 mm; Gate 1 $\rightarrow$ <i>Idle</i> ; fjord unchanged (gate flow = 0)	4483	2520	0
9	<code>c_end_cycle</code>	Phase $\rightarrow$ <i>Env</i>	4483	2520	0
10	<code>u_sea_step</code>	$k \rightarrow 156$, sea = 4523 mm. Sea now exceeds 4500 mm all gates are already closed. G1 is satisfied.	4523	2520	0

This sequence demonstrates the following:

- G1 (absence of forbidden behavior): The state invariant “gates must be closed when the sea level exceeds 4.5 m” is never violated. At step 10 the sea crosses the threshold and every gate has $h = 0$.
- Pre-close policy (presence of required behavior): The requirement automaton `req_no_rise_in_preclose` prevents any `c_gate_rise` from step 4 onward (sea ≥ 4443 mm), giving the supervisor time to close gates while they can still be lowered one step at a time under the G3 actuation-interval constraint.
- G3 (absence of forbidden behavior): Every gate-actuation start (`c_gate_rise` at step 1, `c_gate_lower` at step 5) is followed by a `u_gate_step` before the next start, respecting the 5 s interval modeled by `gate_start_interval`.

The plant is synthesized with initial conditions $k=153$ and $lvl=2500$. To reproduce, open `supervisor.cif` in Eclipse ESCET and run the simulator. Select the events in the order listed above; at each step the simulator only offers transitions that the supervisor permits, confirming that the sequence is a valid supervised sequence of events.

4.2 | Test sequence 2: Gate opening enforced by high fjord during falling sea

This sequence tests the supervisor's response to a different hazard: a fjord that is already near its upper limit while the sea is receding. Whereas test 1 stresses the *closing* policy, test 2 stresses the *opening/draining* policy. The same synthesized supervisor is used; the only difference is the initial state. Two initial values are changed in the plant for this scenario ($k=153 \rightarrow 224$, $lvl=2500$ mm $\rightarrow$ 3700 mm); all automata, guards, invariants and requirements are identical to those used for test 1. The resulting initial state is $k=224$, sea = 3517 mm, $lvl=3700$ mm, all gates closed, no rain, phase = *Decide*, and the policy helper variables evaluate to `fjord_near_high = true`, `sea_falling = true`, `sea_well_below_fjord = true`, so `high_drain_now = true`. Under these conditions the supervisor must open at least two gates before the environment is allowed to advance, so that the fjord is drained rather than rising further.

Table 4.2: Test sequence 2: Gate opening enforced by a high fjord during a falling sea.

#	Event	Effect	Sea	Fjord	h_1	h_2
0	—	Initial state; <code>c_end_cycle</code> is <i>not</i> offered because <code>total_open = 0 < 2</code> while <code>high_drain_now</code>	3517	3700	0	0
1	<code>Gate1.c_gate_rise</code>	Supervisor opens Gate 1; gate enters <i>Opening</i>	3517	3700	0	0
2	<code>Gate1.u_gate_step</code>	Gate 1 height becomes 1 (3 m); gate enters <i>Moving</i> ; <code>c_end_cycle</code> is still not offered (<code>total_open = 1 < 2</code>)	3517	3700	1	0
3	<code>Gate2.c_gate_rise</code>	Supervisor opens Gate 2; gate enters <i>Opening</i>	3517	3700	1	0
4	<code>Gate2.u_gate_step</code>	Gate 2 height becomes 1; gate enters <i>Moving</i> ; <code>total_open = 2</code> , so <code>c_end_cycle</code> is now offered	3517	3700	1	1
5	<code>c_end_cycle</code>	Phase $\rightarrow$ <i>Env</i>	3517	3700	1	1
6	<code>u_sea_step</code>	$k \rightarrow 225$, sea = 3457 mm; Gate 1, 2 $\rightarrow$ <i>Idle</i> ; fjord $-40 \rightarrow 3660$ mm (negative gate flow; 2×20)	3457	3660	1	1
7	<code>c_end_cycle</code>	Phase $\rightarrow$ <i>Env</i>	3457	3660	1	1
8	<code>u_sea_step</code>	$k \rightarrow 226$, sea = 3396 mm; fjord $-40 \rightarrow 3620$ mm	3396	3620	1	1
9	<code>c_end_cycle</code>	Phase $\rightarrow$ <i>Env</i>	3396	3620	1	1
10	<code>u_sea_step</code>	$k \rightarrow 227$, sea = 3335 mm; fjord $-40 \rightarrow 3580$ mm. The fjord has been drained by 120 mm in three environmental steps, without ever exceeding the upper bound.	3335	3580	1	1

This sequence demonstrates the following:

- High-drain policy (presence of required behavior): The invariant “`high_drain_now` implies `total_open` ≥ 2 in phase *Env*” is enforced by the supervisor by forcing at least two gates open before the first `c_end_cycle`. Concretely, `c_end_cycle` is *absent* from the offered transitions at steps 0 and 2 and only becomes available at step 5, once `total_open` = 2.
- Gate-height cap under high drain (absence of forbidden behavior): The invariant “`high_drain_now` implies $h_i \leq 1$ for every gate” keeps both actuated gates at $h = 1$. From step 6 onward, no `c_gate_rise` is offered for Gate 1 or Gate 2 because it would put them at $h = 2$, which is forbidden while `high_drain_now` holds. `Gate3.c_gate_rise` and `Gate4.c_gate_rise` remain offered, confirming that the restriction is based on the height of the gate.
- L1 (absence of forbidden behavior): The fjord level evolves $3700 \rightarrow 3660 \rightarrow 3620 \rightarrow 3580$ mm; the upper bound $lvl \leq 4000$ is respected at every step.
- G3 (absence of forbidden behavior): Every gate-actuation start (`c_gate_rise` at steps 1 and 3) is followed by a `u_gate_step` before the next start, respecting the 5 s interval modeled by `gate_start_interval`.

The plant for this test is synthesized with initial conditions $k=224$ and $lvl=3700$, which lies under `/test_case2/`. To reproduce, open `test_case2/supervisor_case2.cif` in Eclipse ESCET and run the simulator. Select the events in the order listed above; at each step the simulator only offers transitions that the supervisor permits.